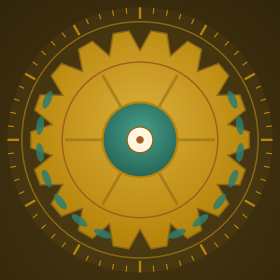

ARDEN · BUILD'O'THON 2026

Trilingual Tales — Build'o'thon Team Assignments

Parallel workstreams for a one-day sprint

ARDEN · 24 JUNE 2026

The Product in One Sentence

A React Native / Expo children's story app delivering Khmer, English, and French narrated tales for the 0–3 developmental band — built for diaspora parents who want a bedtime ritual, not just another reading tool.

Before Teams Split — First 30 Minutes (Whole Group)

These decisions gate every parallel workstream. Do not split until they are locked.

1. Data model consensus — Agree the `Story` and `Page` schema (see Section 6).

2. Supabase project created — Team 3 initialises and shares credentials to all other teams.

3. Demo content agreed — Team 2 confirms which 3 Khmer folk tales will be used, names the audio file format (MP3, 64 kbps), and seeds placeholder rows in the database.

4. Working name — The documented name "Trilingual Tales" is a placeholder. Agree a warm, ownable working name before the demo. It need not be final — it just needs to not be generic.

5. Expo targets confirmed — This build targets iOS, Android, and Expo Web. All teams write no platform-specific code without flagging it.

Team 1 — Core App + Reader

Stack: React Native + Expo SDK

Owns: - Expo project scaffold, navigation structure (Expo Router) - Language picker — KM / EN / FR toggle, persists to user profile - Level/stage picker — filter by L1-L6 or 0-3 developmental band - Story list screen — cover grid with free/locked badges - Story reader — page-turn navigation, illustration display, text per language - Audio playback — Expo AV, page-turn chime between pages - Bedtime mode toggle — warm screen overlay, reduced animation, audio-only option, parental lock (child cannot navigate away) - Expo Web compatibility pass

Do first (minute one):

```
npx create-expo-app trilingual-tales --template blank-typescript
npx expo install @expo-google-fonts/noto-sans-khmer expo-av expo-router
```

Khmer font must be tested on a real device within the first 30 minutes. Emulators do not reliably render Khmer Unicode. If Noto Sans Khmer does not render, resolve it before building any UI.

Integration needs from other teams: - Supabase URL + anon key from Team 3 - Story and Page schema from the agreed data model - Audio file URL format from Team 2 - Color tokens and font pairing from Team 4

Demo success criteria: A parent opens the app, taps French, picks a story at the right stage, hears narration, taps the language button to switch mid-story to Khmer, toggles bedtime mode on. That sequence works end to end.

Team 2 — Content + Audio Pipeline

Stack: Supabase Storage, Google Cloud TTS or ElevenLabs, any scripting language

Owns: - Selection and writing up of 3-5 Khmer folk tales (original texts sourced from scratch) - Full text in all three languages: Khmer original + English translation + French translation - Audio narration files — TTS for demo (Google Cloud TTS or ElevenLabs); real narrator is a production-phase decision - Illustration sourcing — Creative Commons images or AI-generated placeholders clearly marked as placeholders - Upload all assets to Supabase Storage and seed the database - Freemium gate — mark 2 stories as `is_free = true`; remainder locked

Source notes for folk tales: Public-domain Khmer folk tales (e.g., stories from the Jataka tradition in Khmer, traditional animal fables) are the safest IP lane. Team 2 owns confirming that every piece of text is either original or demonstrably public domain.

Audio format agreement (lock at T+0): - Format: MP3 at 64 kbps - One file per page per language - Naming: `{story_id}_{page_number}_{lang}.mp3` — e.g., `001_03_km.mp3`

Integration deliverables: - Supabase seed SQL or CSV to Team 3 by T+1:30 - Final audio files uploaded to Storage by T+3:00 - Working Supabase Storage URLs confirmed with Team 1

Demo success criteria: 3 complete stories in the database. Each story has all pages, all 3 languages of text, and at least 1 page of audio per language. Illustrations present on every page (even placeholders). 2 stories free, 1+ locked.

Team 3 — Backend + Auth

Stack: Supabase (Postgres + Auth + Storage + Edge Functions), TypeScript client

Owns: - Supabase project initialisation and credential distribution (first task — block nothing) - Database schema from the agreed data model (run migrations) - Row-level security: free stories readable by all; premium stories require subscription check - Auth: email/password signup and Google social login - User profile table: preferred language, subscription status - Mock subscription state: a toggle in the user profile (`is_premium: bool`) settable via a simple admin screen or direct Supabase Studio — no real payment flow required - API client export shared with Team 1

Data model (to be ratified by all teams at T+0):

```
-- Stories
create table stories (
  id uuid primary key default gen_random_uuid(),
  title_km text, title_en text, title_fr text,
  level text,              -- L1, L2, L3, L4, L5, L6
  dev_stage text,          -- '0-1', '1-2', '2-3', '3+'
  is_free boolean default false,
  cover_image_url text,
  created_at timestamptz default now()
);

-- Pages
create table pages (
  id uuid primary key default gen_random_uuid(),
  story_id uuid references stories(id),
  page_number int,
  text_km text, text_en text, text_fr text,
  illustration_url text,
  audio_url_km text, audio_url_en text, audio_url_fr text
);

-- User profiles (extends Supabase auth.users)
create table profiles (
  id uuid primary key references auth.users(id),
  preferred_language text default 'en',
  is_premium boolean default false
);
```

Integration deliverables: - Supabase credentials (URL + anon key) to all teams by T+0:15 - Schema migrations run by T+0:30 - Working `supabase-client.ts` export shared with Team 1 by T+0:30 - Admin toggle for `is_premium` available by T+2:00 (so demo can show locked/unlocked states)

Demo success criteria: Sign up works. A user with `is_premium = false` sees locked stories locked. Setting `is_premium = true` (via Studio or admin toggle) unlocks them immediately without app restart.

Team 4 — Design + Brand + Bedtime UX

Owns: - Working name: warm, Khmer-rooted, ideally works across languages. "Trilingual Tales" is a placeholder — a better name should be ready for the demo - One-line tagline in all three languages - Color palette: - Daytime mode: warm ochres, leafy greens, ivory — Cambodian market-stall warmth - Bedtime mode: deep amber, muted brass, near-black — calming and screen-friendly - Typography: Noto Sans Khmer (already in the tech stack) paired with a warm Latin face (Lora or Nunito) - Bedtime mode visual spec delivered to Team 1 as a Figma frame + a written spec of: overlay color/opacity, which animations are disabled, audio-only screen layout - App icon + splash screen (PNG exports at required sizes) - Story cover template — Team 2 uses this for the demo stories - Figma prototype of the full reading flow (supplements the real app in the demo)

Deliverables and timing: - Color tokens to Team 1 by T+0:30 (or they ship with greyscale and swap later) - Story cover template to Team 2 by T+1:00 - Bedtime mode spec to Team 1 by T+1:30 - App icon + splash to Team 1 by T+3:00 - Figma prototype complete by T+4:00

Demo success criteria: The app *feels* premium and calm. A first-time viewer watching the demo does not say "it looks like a prototype." Bedtime mode is visually distinct and noticeably calmer than daytime mode.

Integration Checkpoints

TIME	EVENT
T+0:00	Whole group: data model locked, Supabase project live, working name chosen
T+0:15	Team 3 shares Supabase credentials to all
T+0:30	Teams split — parallel work begins
T+1:30	Team 2 delivers seed SQL; Team 1 confirms Khmer font renders on device
T+2:30	Checkpoint: Team 1 connects to real Supabase data; Team 3 confirms RLS
T+3:00	Team 2 delivers final audio files and URLs
T+3:30	Full integration: real content + auth + bedtime mode working end to end
T+4:00	Team 4 delivers final assets; Figma prototype complete
T+4:30	Demo prep: script the flow, dry run, record a video backup
T+5:00	Demo

The Demo Script

> *A parent opens the app at 8pm. They pick French. They find a Khmer folk tale at their child's stage. They tap play — narration starts in French. They tap the language button and it switches to Khmer mid-story. They switch on bedtime mode — the screen warms, animations calm. They reach the last page and the chime sounds. They try to tap to a locked story — they see the freemium gate. They "subscribe" (admin toggle flipped) — the story unlocks.*

That is the complete demo. No quiz, no coloring page, no teacher dashboard, no hardware device. Those are Phase 2 and 3.

GitHub Repository

Repo: `github.com/michaelwolfsdensigma/trilingual-tales`

All teams commit to this repo. Branch strategy for the day:

```
main          — stable; only merge after integration checkpoints
team/1-app    — Team 1 work
team/2-content — Team 2 scripts + seed data
team/3-backend — Team 3 schema + RLS
team/4-design  — Team 4 assets + Figma exports
```

Merge to `main` at each integration checkpoint. Resolve conflicts together, not alone.

What This Is Not Building Today

To protect scope, the following are explicitly deferred:

- Coloring / activity packs (Phase 1.5)
- Quiz / teacher mode / B2B school portal (Phase 2)
- Physical off-screen device (Phase 3 — separate venture)
- Real payment processing (mock subscription state only)
- Offline download (table stakes, post-demo)
- Grade-aligned curriculum mapping (Phase 3)

If a team member says "what if we added..." during the build — add it to a `FUTURE.md` file in the repo. Do not build it today.

Risks to Watch

Khmer font rendering — test on real hardware in the first 30 minutes. Emulators lie.

Audio latency — pre-load audio on story open; do not start on tap. First-play delay will kill the demo flow.

Scope creep — the whitepaper contains months of features. Today's scope is the demo script above and nothing else.

Content volume — 3 complete stories (all pages, all languages, audio) beats 10 half-finished ones.

– rendered in the Æthel standard –